

MANUAL USUARIO – PROYECTO #1

Miguel Andrés Álvarez López 202406012 – Lenguajes Formales y de Programacion I

Problema #1 como diablos empiezo:

Ya cuando el auxiliar nos brindo una plantilla para trabajar el problema era como diablos empezar, donde declarábamos estructuras al leer?. Donde declaramos para leer las etiquetas? Y demás preguntas

```
#ifndef LEXICALANALYZER_H
#define LEXICALANALYZER_H
//PART 2
#include "Token.h"
#include <string>
#include <vector>
//Estructuras para GUARDAR
struct Medic{//medicos
    string nombre;
    string especialidad;
    string codigo;
    int citas;
    int pacientes;
    string nivelCarga;
};

struct Patient{
    string nombre;
    string edad;
    string blood; //TIPO DE SANGRE
    string habita; //habitacion de hospedaje
};

struct Citas{
    string nombre_p; // aca tambien
    string nombre_dr;
    string fecha;
    string hora;
};

struct Diagnostic{
    string nombre_p; //aca hago mi magia
    string condit; //Condicion
    string medicamento; //MEDICAMENTOS
    string dosis; //Tipo de dosis DIARIA,CADA_N_HORAS
};
```

solución:

Los Archivos header (.h) son muy Utiles, ya que se declara que se existen pero no se declaran o especifica su funcionalidad. Solo hacen saber al programa que existen. Sirvio un montón para declarar las estructuras, Vectores a utilizar, Funciones que leen esos vectores y archivos, etc..

```

class LexicalAnalyzer {
public:

    explicit LexicalAnalyzer(const string& fuente); //Constructor OBLIGATORIO
    //Constante para que no cambie
    //Contenedores ALMACENARAN LOS DATOS AL LEER EL DOCUMENTO
    vector<Medic> medStorage; //
    vector<Citas> citStorage; //
    vector<Patient> patStorage;
    vector<Diagnostic> diagStorage;

    // Retorna todos los tokens del archivo
    vector<Token> tokenize(); //declara vector para tokenizacion

    // Errores acumulados durante el análisis
    struct ErrorLexico {
        string lexema;
        string tipoError;
        string descripcion;
        int linea;
        int columna;
    };
    vector<ErrorLexico> errores;

private:
    //METODOS Y VARIABLES ABSTRACTAS
    string codigo; // Contenido completo del archivo
    int pos; // Posición actual en el string
    int linea; //EN QUE LINEA
    int columna; //EN QUE COLUMNA
    char actual(); // Carácter en pos actual
    char avanzar(); // Avanza y retorna el siguiente carácter
    void saltarBlancos(); // Ignora espacios, tabs, saltos de línea
    Token siguienteToken(); // AFD principal
    Token leerPalabraReservada(); // Reconoce HOSPITAL (y futuras), retorna
    Token leerString(); //Lectura de strings
    Token leerNum(); //lectura de numeros o guiones
    void registrarError(const string& lexema,const string& tipo,const string& desc);

    //REGISTRAR DATOS EN LEXICALANALYZER
    void registrarMedicos(const vector<Token>& tokens, int& pos);
    void registrarCitas(const vector<Token>& tokens, int& pos);
    void registrarPacientes(const vector<Token>& tokens, int& pos);
    void registrarDiagnosticos(const vector<Token>& tokens, int& pos);

};

```

Problema #2 Como diablos leo el contenido y lo asigno a su estructura?:

Para la lectura debía utilizar las funciones brindadas pero para los específicos eran bastante complicados y por cual empezar.

solución:

Por medio de funciones declaradas en lexicalAnalyzer se implementaron en el archivo .h y .cpp para utilización, Implementando una función de Registrar,, para cada componente importante del proyectom extrayendo los tokens generados y leyéndolos según como este estructurado el archivo.

```
//REGISTRAR DATOS EN LEXICALANALYZER
void registrarMedicos(const vector<Token>& tokens, int& pos);
void registrarCitas(const vector<Token>& tokens, int& pos);
void registrarPacientes(const vector<Token>& tokens, int& pos);
void registrarDiagnosticos(const vector<Token>& tokens, int& pos);
```

```
// -- LECTURA DE DATOS --
//!! REGISTRAR DATOS
void LexicalAnalyzer::registrarMedicos (const vector<Token>& tokens, int& pos){
    qDebug() << "DOCTOR!";
    // Verificar que hay una llave de apertura
    if (pos < (int)tokens.size() && tokens[pos].tipo == TokenType::LLAVE_ABRE) {
        pos++; // Saltar '{'
        // Mientras no lleguemos al cierre de la sección
        while (pos < (int)tokens.size() && tokens[pos].tipo != TokenType::LLAVE_CIERRA) {
            // Buscar la palabra clave "medico"
            if (tokens[pos].tipo == TokenType::medico) {
                Medic nuevoMedico;
                pos++; // Saltar "medico"
                if (pos < (int)tokens.size() && tokens[pos].tipo == TokenType::DOS_PUNTOS) {
                    pos++;
                }
                // Leer nombre del médico
                if (pos < (int)tokens.size() && tokens[pos].tipo == TokenType::STRING) {
                    nuevoMedico.nombre = tokens[pos].lexema;
                    qDebug() << "SE AGREGO NOMBRE";
                    pos++;
                }
                if (pos < (int)tokens.size() && tokens[pos].tipo == TokenType::CORCH_ABRE) { //CUANDO DETECTE CORCH_ABRE
                    pos++;
                }
            }
            while (pos < (int)tokens.size() && tokens[pos].tipo != TokenType::CORCH_CIERRA) { //MIENTRAS NO ENCUENTRE CORCH_CIERRA
                //RECONOCE EL TIPO DE ESPECIALIDAD
                if (tokens[pos].tipo == TokenType::especialidad) { //SI ENCUENTRA LA PALABRA RESERVADA
                    pos++; // Saltar

                    if (pos < (int)tokens.size() && tokens[pos].tipo == TokenType::DOS_PUNTOS) {
                        pos++;
                    }

                    // El valor puede ser CARDIOLOGIA, NEUROLOGIA , etc...
                    if (pos < (int)tokens.size() &&
                        tokens[pos].tipo == TokenType::STRING ||
                        tokens[pos].tipo == TokenType::CARDIOLOGIA ||
                        tokens[pos].tipo == TokenType::NEUROLOGIA ||
                        tokens[pos].tipo == TokenType::PEDIATRIA ||
                        tokens[pos].tipo == TokenType::CIRUGIA ||
                        tokens[pos].tipo == TokenType::MEDICINA_GENERAL ||
                        tokens[pos].tipo == TokenType::ONCOLOGIA){
                        nuevoMedico.especialidad = tokens[pos].lexema; //ASIGNA LA ESPECIALIDAD
                    }
                }
            }
        }
    }
}
```

```

//CREACION DE TOKENS
vector<Token> LexicalAnalyzer::tokenize() {
    vector<Token> tokens; //invoca al vector de tokens de LexicaAnalyzer
    pos = 0; linea = 1; columna = 1;

    //Limpieza de vectores
    errores.clear();
    medStorage.clear();
    diagStorage.clear();
    patStorage.clear();
    citStorage.clear();
    //Parseo GENERAL
    while (true) {
        Token t = siguienteToken(); //lee os tokens
        if (t.tipo == TokenType::FIN_ARCHIVO) break;
        if (t.tipo != TokenType::DESCONOCIDO)
            tokens.push_back(t);
    }
    //Parseo de Medicos, GENERA LOS TOKENS PARA LOS VECTORES
    int posTok = 0;
    while(posTok < (int)tokens.size()){
        if(tokens[posTok].tipo == TokenType::MEDICOS){
            posTok++;
            registrarMedicos(tokens,posTok);
        }else if(tokens[posTok].tipo == TokenType::PACIENTES){
            posTok++;
            registrarPacientes(tokens,posTok);
        }else if(tokens[posTok].tipo == TokenType::CITAS){
            posTok++;
            registrarCitas(tokens,posTok);
        }else if(tokens[posTok].tipo == TokenType::DIAGNOSTICOS){
            posTok++;
            registrarDiagnosticos(tokens,posTok);
        }else{
            posTok++;
        }
    }

    return tokens;//retorno
}

```

Problema #3 Como Implementar la generación de Archivos HTML

El principal problema para esta sección era como generar un archivo HTML que pudiera iterar el contenido almacenado en los vectores y generar el archivo en si

solución:

Por medio de funciones declaradas en mainwindow, se implementaron en el archivo .cpp para su posterior uso y definición de estructuración.

```
private slots: //todo los metodos que responden eventos
    void cargarArchivo();
    void analizarArchivo();
    void abrirReporte1();
    void abrirReporte2();
    void abrirReporte3();
    void abrirReporte4();
    void abrirReporte5(); //TOKENS //GENERAR REPORTE

    // RETORNO DE VALORES
    int returnMeds();
    int returnPacient();
    int returnCita();
    int returnDiag();
    double returnProm();
    string mayorCarga();💡
    string mayorPreS(); //Medicamento con mayor prescritura💡

private:

    //GENERACION
    //CONTENEDOR PARA VECTORES DE LEXICALANALYZER
    vector<Medic> medicos; //MEDICOS
    vector<Patient> pacientes;
    vector<Citas> cita;
    vector<Diagnostic> diagnosticos;
    vector<ErrorLexico> lexicalError;
    //SE AGREGARAN MAS EN EL FUTURO

    //REPORTES
    QString clientHist(); //HISTORIAL DE PACIENTES

    QString reportMed(); // EL MEDICO Y SU TIPO DE CARGA
    QString citasReport(); //REPORTE DE CITAS Y POSIBLES CONFLICTOS
    QString hospitalStats(); // REPORTE GENERAL (EASIEST)
    QString lexicalErrors();
    QString dotFile();
```

```

void MainWindow::abrirReporte4() { // REPORTE GENERAL

    //GENERAR REPORTE PACIENTES
    //REPORTE DE CITAS

    //GENERAR REPORTE
    qInfo() << "Generando reporte General Hospital + Archivo.dot ";

    if (archivoActual.isEmpty()) {
        qWarning() << "ERROR: No hay archivo cargado";
        QMessageBox::warning(this, "Error", "Primero carga un archivo .med");
        return;
    }

    if(!archivoActual.isEmpty()){

        QString ruta = QFileInfo(archivoActual).absolutePath();
        QString dotPath = ruta + "/jerarquiaHospital.dot"; //Ruta donde se almacenara el archivo.dot
        QString RePath = ruta + "/reporte_g_hospital.html"; //Ruta donde se almacenara reporte de Hospital

        QDir dir(ruta);
        if(!dir.exists()){
            QMessageBox::critical(this, "Error", "La ruta: " + ruta + " No existe");
            return; //defunde la fucnbion
        }

        QString dotContents = dotFile();
        QString htmlContents = hospitalStats(); //genera el reporte con datos
        QFile archivo(RePath); //guarda el archivo
        QFile archivoDot(dotPath); //guarda archivo dot
        if(archivo.open(QIODevice::WriteOnly | QIODevice::Text)){
            QTextStream out(&archivo);
            out << htmlContents;
            archivo.close();
            //POPUP
            QMessageBox::information(this, "Historial Pacientes!", "Reporte de Historial de Pacientes Generado correctamente correctamente.");
        }else{
            QMessageBox::information(this, "ERROR, Historial Pacientes!", "Ocurrio un error al GENERAR el archivo");
        }

        if(archivoDot.open(QIODevice::WriteOnly | QIODevice::Text)){
            QTextStream out(&archivoDot);
            out << dotContents;
            archivoDot.close();
            //POPUP
            QMessageBox::information(this, "Jerarquia de hospital!", "Reporte de Jerarquia de hospital Generado correctamente correctamente.");
        }else{
            QMessageBox::information(this, "ERROR, Jerarquia de hospital!", "Ocurrio un error al GENERAR el archivo de \n Jerarquia de hospital ");
        }
    }
}

```

Problema #4 Tomar valores sin repetirse y contar valores según el dato

Para algunas funciones o instrucciones para el programa era necesario recorrer los vectores y tomar los valores únicos, pero sin repetirse además de algunas partes del proyecto pedían implementar funciones para contar cuantos médicos estaban por categoría sabiendo que hay mas de 1 categoria existente.

solución:

Por medio de la función `SET<>` que guarda valores únicos sin repetirse y `map<>` para conteo según su valor de identificación se logró solucionar el problema

```
string MainWindow::mayorPres(){
    string mayorPres;
    int maxFrec = 0;
    //TOCA USAR MAP ME CAGO EN TODO
    if(diagnostics.empty()){
        return "NO HAY MEDICAMENTOS REGISTRADOS";
    }
    //mapa sin orden
    //Almacena datos sin ordenarlos
    unordered_map<string,int> frecuenciaMed;

    for(int d=0; d<diagnostics.size();d++){
        if(!diagnostics[d].medicamento.empty()){ //si no esta vacio el espacio de medicamento
            frecuenciaMed[diagnostics[d].medicamento]++;
        }
    }

    if(frecuenciaMed.empty()){
        return "NO HAY MEDICAMENTOS REGISTRADOS";
    }

    //foreach para los dos datos
    for(const auto& [medicamento,frecuencia]: frecuenciaMed){
        if(frecuencia > maxFrec){
            maxFrec = frecuencia;
            mayorPres = medicamento;
        }
    }
    return mayorPres + " || Veces Recetada: " + to_string(maxFrec);
}
```